```
#include <stdio.h>
int main() {
   int n, i;
   printf("How many processes? ");
   scanf("%d", &n);
   int burst[n], wait[n], turn[n];
   for(i = 0; i < n; i++) {
      printf("Burst time for P%d: ", i+1);
      scanf("%d", &burst[i]);
   }
   wait[0] = 0;
   for(i = 1; i < n; i++)
      wait[i] = wait[i-1] + burst[i-1];
   printf("\nP\tBurst\tWait\tTurnaround\n");
   float avgW=0, avgT=0;
   for(i = 0; i < n; i++) {
      turn[i] = wait[i] + burst[i];
      avgW += wait[i]; avgT += turn[i];
      printf("P%d\t%d\t%d\t%d\n", i+1, burst[i], wait[i], turn[i]);
   }
   printf("Avg Wait=%.2f  Avg Turnaround=%.2f\n", avgW/n, avgT/n);
   return 0;
}
```

```
#include <stdio.h>
int main() {
   int n, i, j, temp;
   printf("How many processes? ");
   scanf("%d", &n);

   int pid[n], burst[n], wait[n], turn[n];
```

```
    for(i = 0; i < n; i++) {
        pid[i] = i+1;
        printf("Burst time for P%d: ", i+1);
        scanf("%d", &burst[i]);
    }
    for(i = 0; i < n-1; i++)
        for(j = 0; j < n-i-1; j++)
            if(burst[j] > burst[j+1]) {
                temp=burst[j]; burst[j]=burst[j+1]; burst[j+1]=temp;
                temp=pid[j];   pid[j]=pid[j+1];     pid[j+1]=temp;
            }
    wait[0] = 0;
    for(i = 1; i < n; i++)
        wait[i] = wait[i-1] + burst[i-1];
    printf("\nP\tBurst\tWait\tTurnaround\n");
    float avgW=0, avgT=0;
    for(i = 0; i < n; i++) {
        turn[i] = wait[i] + burst[i];
        avgW += wait[i]; avgT += turn[i];
        printf("P%d\t%d\t%d\t%d\n", pid[i], burst[i], wait[i], turn[i]);
    }
    printf("Avg Wait=%.2f  Avg Turnaround=%.2f\n", avgW/n, avgT/n);
    return 0;
}
```

```
#include <stdio.h>
int main() {
    int n, i, j, temp;
    printf("How many processes? ");
    scanf("%d", &n);
```

```
    int pid[n], burst[n], priority[n], wait[n], turn[n];
    for(i = 0; i < n; i++) {
        pid[i] = i+1;
        printf("Burst & Priority for P%d: ", i+1);
        scanf("%d %d", &burst[i], &priority[i]);
    }

    for(i = 0; i < n-1; i++)
        for(j = 0; j < n-i-1; j++)
            if(priority[j] > priority[j+1]) {
                temp=priority[j]; priority[j]=priority[j+1]; priority[j+1]=temp;
                temp=burst[j];    burst[j]=burst[j+1];       burst[j+1]=temp;
                temp=pid[j];      pid[j]=pid[j+1];           pid[j+1]=temp;
            }

    wait[0] = 0;
    for(i = 1; i < n; i++)
        wait[i] = wait[i-1] + burst[i-1];

    printf("\nP\tBurst\tPri\tWait\tTurnaround\n");
    float avgW=0, avgT=0;
    for(i = 0; i < n; i++) {
        turn[i] = wait[i] + burst[i];
        avgW += wait[i]; avgT += turn[i];
        printf("P%d\t%d\t%d\t%d\t%d\n", pid[i], burst[i], priority[i], wait[i], turn[i]);
    }
    printf("Avg Wait=%.2f  Avg Turnaround=%.2f\n", avgW/n, avgT/n);
    return 0;
}
```

#include <stdio.h>

```
int main() {
    int bSize[] = {100, 500, 200, 300, 600}; // memory blocks
    int pSize[] = {212, 417, 112, 426};      // process sizes
    int nb = 5, np = 4;
    int alloc[np];

    for(int i = 0; i < np; i++) alloc[i] = -1;

    for(int i = 0; i < np; i++) {
        int best = -1; // index of best block
        for(int j = 0; j < nb; j++) {
            if(bSize[j] >= pSize[i]) {         // block fits
                if(best == -1 || bSize[j] < bSize[best]) // smallest fit
                    best = j;
            }
        }
        if(best != -1) {
            alloc[i] = bSize[best];
            bSize[best] -= pSize[i]; // reduce block size
        }
    }

    printf("Process\tSize\tBlock Allocated\n");
    for(int i = 0; i < np; i++) {
        if(alloc[i] != -1)
            printf("P%d\t%d\t%d\n", i+1, pSize[i], alloc[i]);
        else
            printf("P%d\t%d\tNot Allocated\n", i+1, pSize[i]);
    }
    return 0;
}
```

```
#include <stdio.h>
int main() {
   int bSize[] = {100, 500, 200, 300, 600};
   int pSize[] = {212, 417, 112, 426};
   int nb = 5, np = 4;
   int alloc[np];

   for(int i = 0; i < np; i++) alloc[i] = -1;

   for(int i = 0; i < np; i++) {
      for(int j = 0; j < nb; j++) {
         if(bSize[j] >= pSize[i]) { // first block that fits
            alloc[i] = bSize[j];
            bSize[j] -= pSize[i];
            break;            // stop after first fit!
         }
      }
   }

   printf("Process\tSize\tBlock Allocated\n");
   for(int i = 0; i < np; i++) {
      if(alloc[i] != -1)
         printf("P%d\t%d\t%d\n", i+1, pSize[i], alloc[i]);
      else
         printf("P%d\t%d\tNot Allocated\n", i+1, pSize[i]);
   }
   return 0;
}
```

```
#include <stdio.h>
int main() {
```

```
    int bSize[] = {100, 500, 200, 300, 600};
    int pSize[] = {212, 417, 112, 426};
    int nb = 5, np = 4;
    int alloc[np];

    for(int i = 0; i < np; i++) alloc[i] = -1;

    for(int i = 0; i < np; i++) {
        int worst = -1;
        for(int j = 0; j < nb; j++) {
            if(bSize[j] >= pSize[i]) {          // block fits
                if(worst == -1 || bSize[j] > bSize[worst]) // largest fit
                    worst = j;
            }
        }
        if(worst != -1) {
            alloc[i] = bSize[worst];
            bSize[worst] -= pSize[i];
        }
    }

    printf("Process\tSize\tBlock Allocated\n");
    for(int i = 0; i < np; i++) {
        if(alloc[i] != -1)
            printf("P%d\t%d\t%d\n", i+1, pSize[i], alloc[i]);
        else
            printf("P%d\t%d\tNot Allocated\n", i+1, pSize[i]);
    }
    return 0;
}
```

```
#include <stdio.h>

int main() {
    int frame[3] = {-1, -1, -1};
    int ref[] = {1, 2, 3, 1, 1, 5, 3,4,6,2};
    int time[3] = {0}, pageFaults = 0, t = 0;

    for (int i = 0; i < 10; i++) {
        int found = 0;

        for (int j = 0; j < 3; j++) {
            if (frame[j] == ref[i]) {
                found = 1;
                time[j] = ++t;
            }
        }

        if (!found) {
            int pos = 0;
            for (int j = 1; j < 3; j++)
                if (time[j] < time[pos]) pos = j;

            frame[pos] = ref[i];
            time[pos] = ++t;
            pageFaults++;
        }

        // Output like you want
        printf("Page %d -> Frames: ", ref[i]);
        for (int j = 0; j < 3; j++)
            frame[j] == -1 ? printf("- ") : printf("%d ", frame[j]);
```

```
        printf(found ? "(hit)\n" : "(FAULT)\n");
    }

    printf("Total Page Faults: %d\n", pageFaults);
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, head, i;
    printf("Initial head position: "); scanf("%d", &head);
    printf("Number of requests: ");   scanf("%d", &n);

    int req[n];
    printf("Enter requests: ");
    for(i = 0; i < n; i++) scanf("%d", &req[i]);

    int totalMovement = 0, current = head;

    printf("\nHead Movement: %d", current);
    for(i = 0; i < n; i++) {
        int move = abs(current - req[i]);
        totalMovement += move;
        current = req[i];
        printf(" -> %d", current);
    }

    printf("\nTotal Head Movement: %d\n", totalMovement);
    return 0;
}
```